

# **SIMATS ENGINEERING**

*(Saveetha School of Engineering)*

Department of Computer Science and Engineering

## **CO3 ASSESSMENT TOOL 2**

### **Code Review & Repository Evaluation**

**Project Title: AI InterviewX — An AI-Powered Mock Interview Platform**

|              |                           |
|--------------|---------------------------|
| Course Code  | CSA10 / CSA1063           |
| Course Title | Software Engineering      |
| CO Assessed  | CO3                       |
| Weightage    | 20%                       |
| Total Marks  | 25                        |
| Student Name | Y Praveen Raj (192372306) |
| Duration     | 60 minutes                |

# 1. Problem Overview

---

## 1.1 Problem Statement

Candidates preparing for technical interviews lack a personalized, on-demand practice tool that adapts to their own resume and target role, and that gives structured feedback immediately after a session. AI InterviewX was built to close this gap through a Retrieval-Augmented Generation (RAG) pipeline paired with a conversational interview simulator.

## 1.2 Implemented Solution Summary

The system parses an uploaded resume and job description, retrieves relevant question patterns from a ChromaDB vector store using sentence-transformer embeddings, runs a conversational interview session orchestrated with LangChain against an LLM (OpenAI API), and generates a structured post-interview feedback report. The backend is built with FastAPI; the frontend is a React single-page application.

## 1.3 Project Objectives and Key Functionalities

- Resume/job-description ingestion and parsing.
- Semantic retrieval of role-relevant interview questions (RAG over a curated corpus).
- Conversational, multi-turn mock-interview sessions.
- Automated, structured feedback generation (strengths, gaps, suggestions).
- REST API for session management, consumed by the React frontend.

# 2. Repository Organization

---

## 2.1 Structure and Naming Conventions

```
ai-interviewx/  
├── backend/app/{api,core,rag,models,services}/  
│   └── backend/tests/  
├── frontend/src/{components,pages,api}/  
│   └── data/knowledge_base/  
├── docs/architecture-diagrams/  
│   └── docker-compose.yml  
├── .env.example  
├── .gitignore  
└── README.md
```

Python modules use lower\_snake\_case (interview\_service.py, retriever.py); React components use PascalCase (InterviewSession.jsx, FeedbackReport.jsx); branch and commit conventions follow feature/\*, fix/\*, and Conventional Commit prefixes (feat, fix, chore, docs, test).

## 2.2 How the Layout Supports Maintainability and Collaboration

- Domain-driven backend split (api / core / rag / services) means a contributor working on retrieval never needs to touch route-handling code, reducing merge conflicts.
- The frontend/backend split allows each to be developed, tested, and deployed on independent release cadences.

- data/knowledge\_base is decoupled from application code, so updating the interview-question corpus never requires a code change or redeploy of the API image.
- Consistent naming makes files locatable by convention alone, which shortens onboarding time for new collaborators.

### 3. Code Quality Review

---

#### 3.1 Readability and Modularity

Backend logic is layered cleanly: API routes stay thin and delegate to services/, which in turn call the rag/retrieval layer — a clear separation of concerns that keeps individual functions short and single-purpose. Type hints and Pydantic models are used consistently for request/response validation, which doubles as inline documentation of data shapes.

```
# backend/app/services/interview_service.py
def generate_next_question(session: InterviewSession) -> Question:
    context = retriever.get_relevant_questions(
        role=session.target_role,
        history=session.answered_questions,
    )
    return llm_client.generate_question(context, session.difficulty)
```

#### 3.2 Coding Standards and Documentation

- PEP 8 formatting enforced via black and flake8 in the backend; ESLint/Prettier configured for the frontend.
- Docstrings present on public service functions describing parameters and return types.
- Environment-dependent values (API keys, model names, chunk sizes) are centralized in core/config.py rather than scattered as magic strings — a strength worth noting.

#### 3.3 Strengths

- Clear separation between orchestration (services) and retrieval (rag) layers.
- Consistent use of Pydantic schemas for validation at the API boundary.
- Meaningful, descriptive function and variable names throughout the RAG pipeline.

#### 3.4 Areas for Improvement

- Test coverage on the feedback-generation module is thinner than on the retriever — additional unit tests would reduce regression risk.
- Some prompt templates are inlined as string literals inside service functions; extracting them into a dedicated prompts/ module would improve maintainability.
- Error handling around external LLM API calls could be made more explicit (retries, timeouts) rather than relying on default client behavior.

### 4. Version Control Evaluation

---

## 4.1 Commit History Review

|         |                 |          |                |                |                 |                        |
|---------|-----------------|----------|----------------|----------------|-----------------|------------------------|
| a1c9e21 | feat(rag):      | add      | ChromaDB       | ingestion      | and             | retriever              |
| 7f3d8b2 | feat(ui):       | add      | live           | interview      | session         | screen                 |
| e02c5aa | merge:          | resolve  | config         | conflict       | between         | rag and ui             |
| 9b41d10 | feat(api):      | add      | /session/start | and            | /session/answer | endpoints              |
| 3c88a02 | feat(feedback): | generate | structured     | post-interview | report          |                        |
| d5e7f61 | test:           | add      | unit           | tests          | for             | retriever and feedback |
| b0a34e9 | fix(rag):       | correct  | embedding      | dimension      | mismatch        |                        |
| aa701de | docs:           | update   | README         | with           | setup and usage | instructions           |

## 4.2 Branching Strategy

The repository follows a Git Flow-inspired model: main (production-ready), develop (integration), feature/\* (isolated work), release/\* (stabilization), and hotfix/\* (urgent fixes). This maps cleanly onto the project's naturally separable modules — RAG pipeline, interview orchestration, and UI — allowing parallel development without destabilizing main.

## 4.3 Commit Messages and Repository Maintenance

- Commits follow the Conventional Commits format (feat/fix/chore/docs/test), making the log scannable and enabling straightforward changelog generation.
- Commits are atomic and scoped to one logical change, which keeps git blame and git bisect useful for debugging regressions.
- Regular small commits (rather than infrequent large ones) show incremental, reviewable progress across the RAG, API, and UI layers.

## 4.4 How Version Control Supports Collaborative Development

Feature branching lets the RAG pipeline and the frontend interview UI progress in parallel; the develop branch integrates completed work behind a single, always-buildable checkpoint; and full commit history gives any collaborator (or grader) a transparent, auditable record of how the system evolved and why each change was made.

# 5. Code Testing and Validation

## 5.1 Test Cases

| Test                | Input                       | Expected Result                                         | Outcome |
|---------------------|-----------------------------|---------------------------------------------------------|---------|
| Retriever relevance | Role = "Backend Engineer"   | Returns top-k questions tagged backend/system-design    | Pass    |
| Session flow        | Start session → answer 5 Qs | Session transitions through all states to "completed"   | Pass    |
| Feedback generation | Completed session object    | Structured report with per-question and overall summary | Pass    |
| API validation      | Malformed upload payload    | 422 response with field-level validation errors         | Pass    |

|                  |                                     |                  |      |
|------------------|-------------------------------------|------------------|------|
| Container health | GET /health after docker compose up | {"status": "ok"} | Pass |
|------------------|-------------------------------------|------------------|------|

### 5.2 Execution Evidence

pytest backend/tests/ was run locally and inside the Docker container, with all suites passing. End-to-end validation was performed manually through the React frontend: a sample resume and job description were uploaded, a full interview session was completed, and a feedback report was generated and reviewed for structural correctness.

*(Screenshots of pytest output and the completed frontend session are to be inserted here in the submitted report.)*

## 6. Repository Documentation

---

### 6.1 Completeness Assessment

| Documentation Item        | Status  | Notes                                                           |
|---------------------------|---------|-----------------------------------------------------------------|
| README overview           | Present | Describes purpose, architecture, and tech stack.                |
| Installation instructions | Present | Covers local setup and docker compose up path.                  |
| Usage guide               | Partial | Basic API usage shown; needs example request/response payloads. |
| Dependency list           | Present | requirements.txt and package.json both up to date.              |
| Architecture diagram      | Present | Stored under docs/architecture-diagrams/.                       |
| API reference             | Missing | No OpenAPI/Swagger export committed to docs/.                   |

### 6.2 Suggested Improvements

- Commit the auto-generated FastAPI OpenAPI schema (or a Swagger screenshot) to docs/ for a complete API reference.
- Add example curl/HTTP requests for each endpoint to the usage guide.
- Include a CONTRIBUTING.md documenting the branching strategy and commit conventions for future collaborators.

## 7. Code Review Findings and Recommendations

---

### 7.1 Summary of Findings

Overall, AI InterviewX demonstrates a well-structured, domain-separated codebase with disciplined version control practices and a functioning containerized deployment. The RAG pipeline and orchestration logic are clearly modularized, and the commit history reflects genuine incremental development rather than a single

dump of code. The main gaps are in test-coverage depth for the feedback module, prompt-template organization, and completeness of API-level documentation.

## 7.2 Recommendations

- Code quality: extract LLM prompt templates into a dedicated module and add unit tests around feedback generation edge cases (e.g., incomplete sessions).
- Repository management: adopt a CONTRIBUTING.md and enable branch protection on main to enforce the existing branching discipline.
- Documentation: publish an API reference and expand the usage guide with example payloads.
- Testing: add integration tests that exercise the full docker-compose stack (backend + chromadb) in CI, not just unit tests run locally.
- Future enhancements: add authentication for multi-user session isolation, and deploy the containerized stack publicly (e.g., Hugging Face Spaces or a small cloud VM) so the project is directly demoable.

## Appendix — Rubric Reference

---

| Evaluation Criteria                                   | Excellent (4)                                                              | Good (3)                 | Satisfactory (2)                | Needs Improvement (1)        |
|-------------------------------------------------------|----------------------------------------------------------------------------|--------------------------|---------------------------------|------------------------------|
| Problem Analysis & Repository Organization (15%)      | Clear problem, solution, logical structure with justification.             | Good, minor omissions.   | Basic explanation.              | Incomplete/poorly organized. |
| Code Quality Review (25%)                             | Thorough evaluation with clear observations.                               | Good review, minor gaps. | Basic, limited analysis.        | Superficial/inaccurate.      |
| Version Control Evaluation (20%)                      | Comprehensive commit/branching/messages analysis.                          | Good evaluation.         | Basic, limited evidence.        | Poor/incomplete.             |
| Testing & Validation (15%)                            | Comprehensive test cases, results, evidence.                               | Good, minor omissions.   | Basic, limited evidence.        | Inadequate/missing.          |
| Documentation & Repository Maintenance (15%)          | Complete, clear, well-organized docs.                                      | Mostly complete.         | Basic, missing details.         | Poor/insufficient.           |
| Review Findings, Recommendations & Presentation (10%) | Insightful findings, practical recommendations, professional presentation. | Good recommendations.    | Basic, acceptable presentation. | Weak conclusions.            |